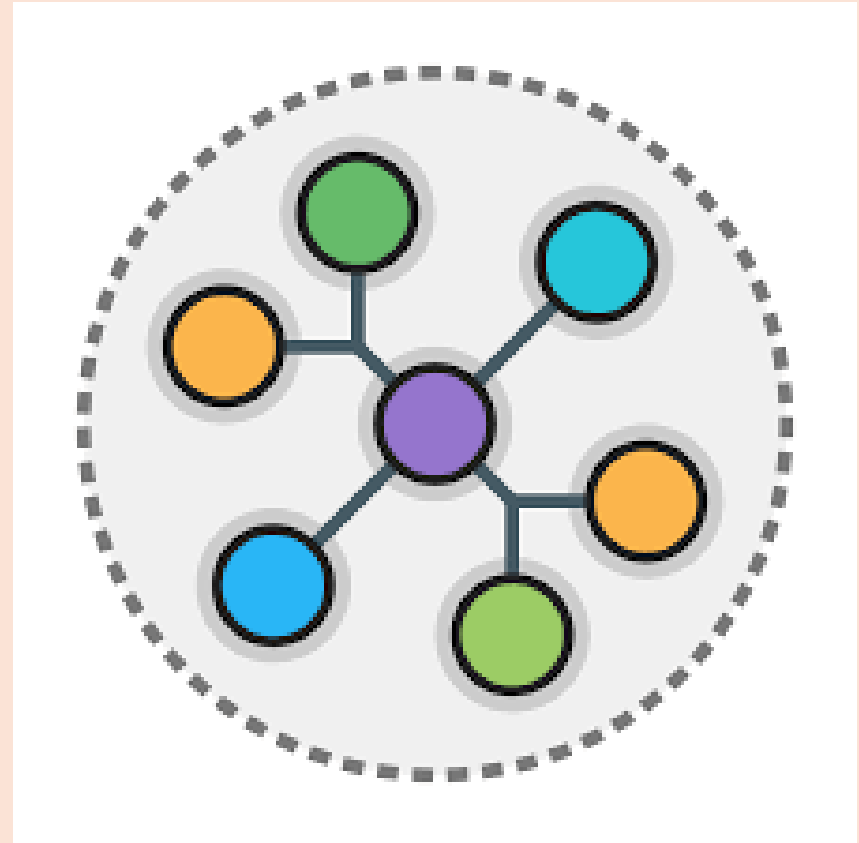


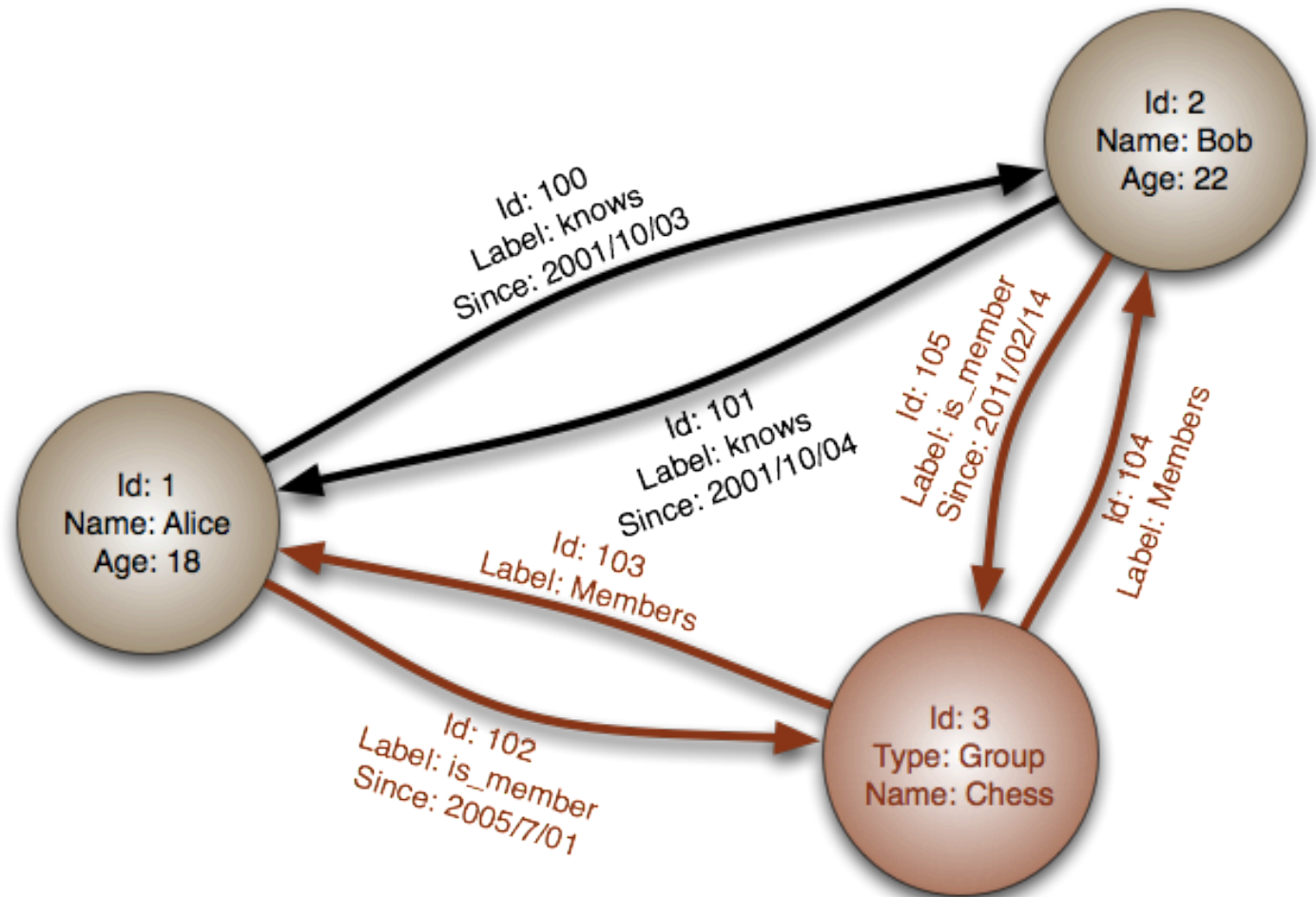
Bases de Datos Orientadas a Grafos

Base de Datos 2



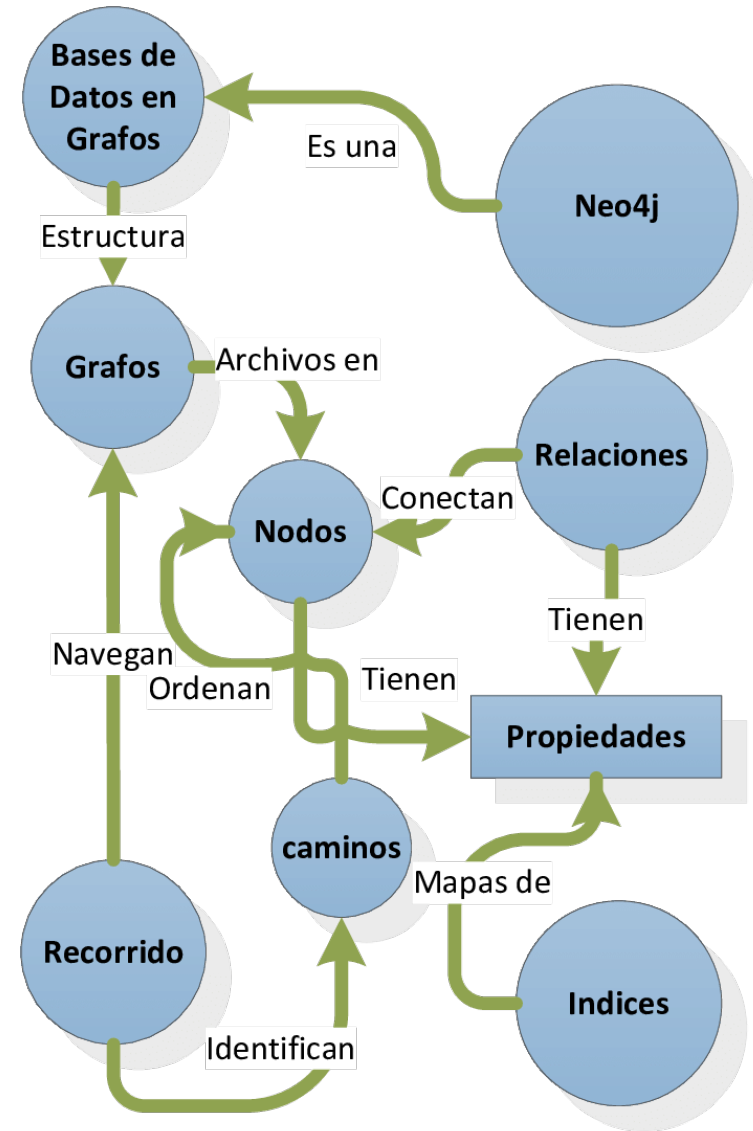
Bases de Datos orientadas a Grafos

- Basada en estructura de grafos (nodos y aristas) para representar y almacenar la información.
- Permiten un fácil almacenamiento de las entidades y las relaciones entre estas.
- Las entidades se representan a través de nodos con sus propiedades.
- Las aristas (arcos) son las relaciones entre los nodos y también pueden tener propiedades.



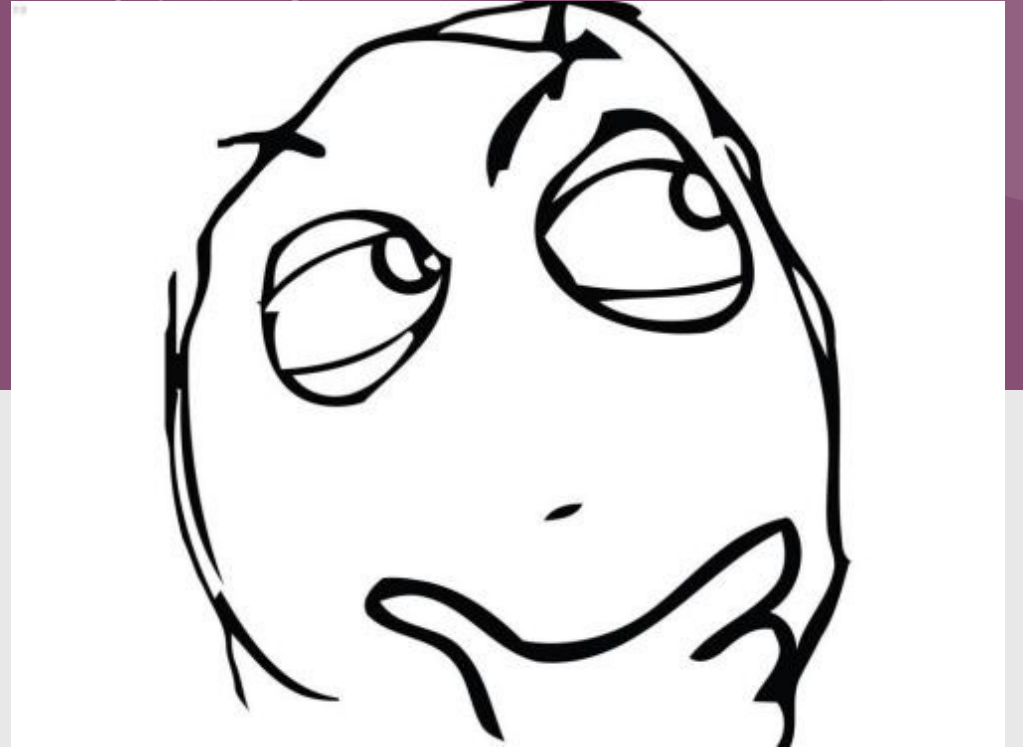
Bases de Datos Orientadas a Grafos

- Podemos hacer una analogía entre un Nodo y un objeto instanciado en una aplicación.
- Los arcos, aristas o relaciones tienen dirección.
- Los nodos están vinculados por relaciones que nos permiten encontrar patrones (trabaja_en, casado_con, etc.).
- La organización del grafo nos permite que los datos sean almacenados una vez y luego interpretados de distintas maneras basándonos en sus relaciones



Para que usamos Grafos?

- Redes sociales (¿quién conoce a quién?)
- Recomendaciones (usuarios que vieron X también vieron Y)
- Fraude financiero (transacciones relacionadas con otras personas o cuentas)
- Supply chain, biología, logística, rutas, etc.



Ventajas

Ventaja	Explicación
Consultas intuitivas	Navegar relaciones se parece a cómo pensamos los problemas
Rendimiento	Mejor rendimiento en consultas de relaciones complejas (vs SQL con muchos JOINS)
Flexibilidad	No necesita esquema rígido, se adapta a datos heterogéneos
Semántica rica	Las relaciones también tienen significado y atributos
Ideal para grafos grandes	Escalables para redes sociales, logística, detección de fraudes, etc.

“Navegar relaciones se parece a cómo pensamos los problemas”

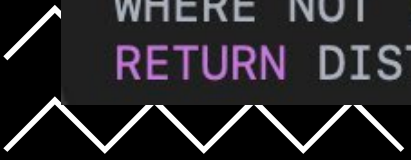
En una base de datos de grafos, no necesitamos pensar en términos de “unir tablas por claves” como en SQL.

Simplemente **seguimos relaciones**, como lo haríamos naturalmente en la vida real.

¿Qué películas le gustaron a los amigos de Ana?”

En la vida real lo pensás así:

“Ana tiene amigos → esos amigos vieron películas → quiero saber cuáles les gustaron”

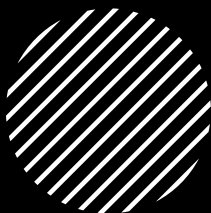


```
MATCH (ana:User {name: "Ana"})-[:FRIENDS_WITH]->(friend)-[:RATED]->(movie)
WHERE NOT (ana)-[:RATED]->(movie)
RETURN DISTINCT movie.title, friend.name;
```

Como es en Neo4j?

Traducción natural:

- Ana está conectada a sus amigos
- Sus amigos están conectados a películas que calificaron
- Filtramos las que Ana no calificó aún
- ¿Cómo sería en una base relacional (SQL)?
 - Tendrías que hacer al menos **3 JOINS** entre:
 - La tabla users
 - La tabla friendships
 - La tabla ratings
 - Y posiblemente una subconsulta para excluir lo que Ana ya calificó
 - Lo que resulta menos natural y más difícil de leer.



Modelo Interno (Native Graph Storage)

Las bases como **Neo4j** usan un almacenamiento **nativo de grafos**, que significa:

- Los **nodos** y **relaciones** se almacenan como estructuras separadas con punteros directos.
- Cada nodo contiene:
 - Un identificador
 - Etiquetas (labels)
 - Propiedades (clave-valor)
 - Punteros a las relaciones entrantes y salientes
- Cada relación contiene:
 - Tipo de relación (:FRIENDS_WITH, :RATED, etc.)
 - Identificador del nodo origen y destino
 - Propiedades propias (por ejemplo, since: 2020)
- **Resultado:** navegar de un nodo a otro es muy rápido (solo sigue los punteros), sin necesidad de hacer joins complejos.

Indexación y Acceso

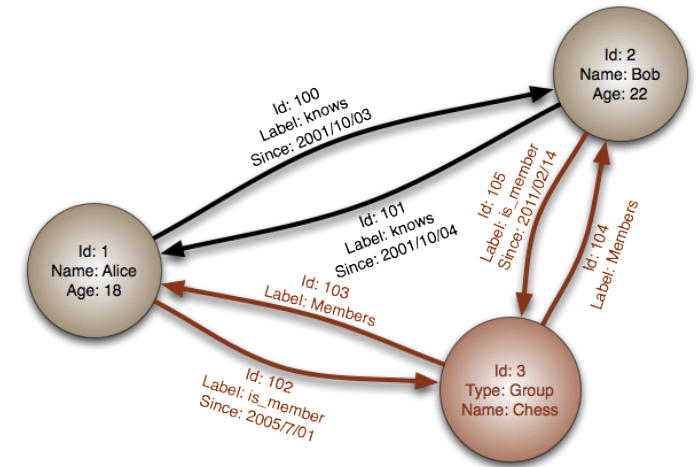
- Los índices se utilizan para **encontrar nodos rápidamente** por una propiedad (name, id, etc.)
- Pero una vez localizado el nodo, **la navegación por relaciones es directa** y no necesita índices adicionales.
- Soporta índices de tipo B-Tree o Hash para propiedades específicas (según la base).

Motor de Consulta (Query Engine)

Usa lenguajes como **Cypher** o **Gremlin**. El motor:

- Interpreta el patrón de consulta (ej: (a)-[:FRIEND]->(b))
- Localiza el nodo inicial (vía índice si hay WHERE)
- Explora el grafo en memoria, siguiendo aristas según el patrón
- Devuelve el resultado del recorrido

ACID??



Cuando NO usarlas

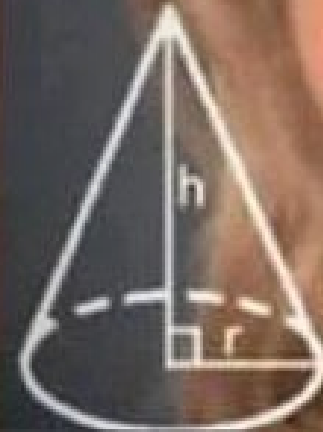
- Sistemas que requieren de actualizaciones masivas sobre todas las entidades o un conjunto de entidades para un atributo o conjunto de atributos específicos.
- Sistemas que requieren una alta distribución de datos debido a su gran tamaño

Please don't

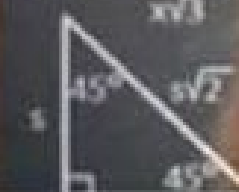


Pero si
hablamos de
relaciones por
que no un
RDBMS???

$$V = \frac{1}{3} \pi r^2 \cdot h$$



	30°	45°	60°
sin	$\frac{1}{2}$	$\frac{\sqrt{2}}{2}$	$\frac{\sqrt{3}}{2}$
cos	$\frac{\sqrt{3}}{2}$	$\frac{\sqrt{2}}{2}$	$\frac{1}{2}$
tan	$\frac{1}{\sqrt{3}}$	1	$\sqrt{3}$



$$= ax^2 + bx + c$$

$$x_1, x_2 = \frac{-b \pm \Delta}{2a}$$

$$\Delta = \sqrt{b^2 - 4ac}$$



Las BDs relacionales no guardan las relaciones entre los datos, solo tablas con id.

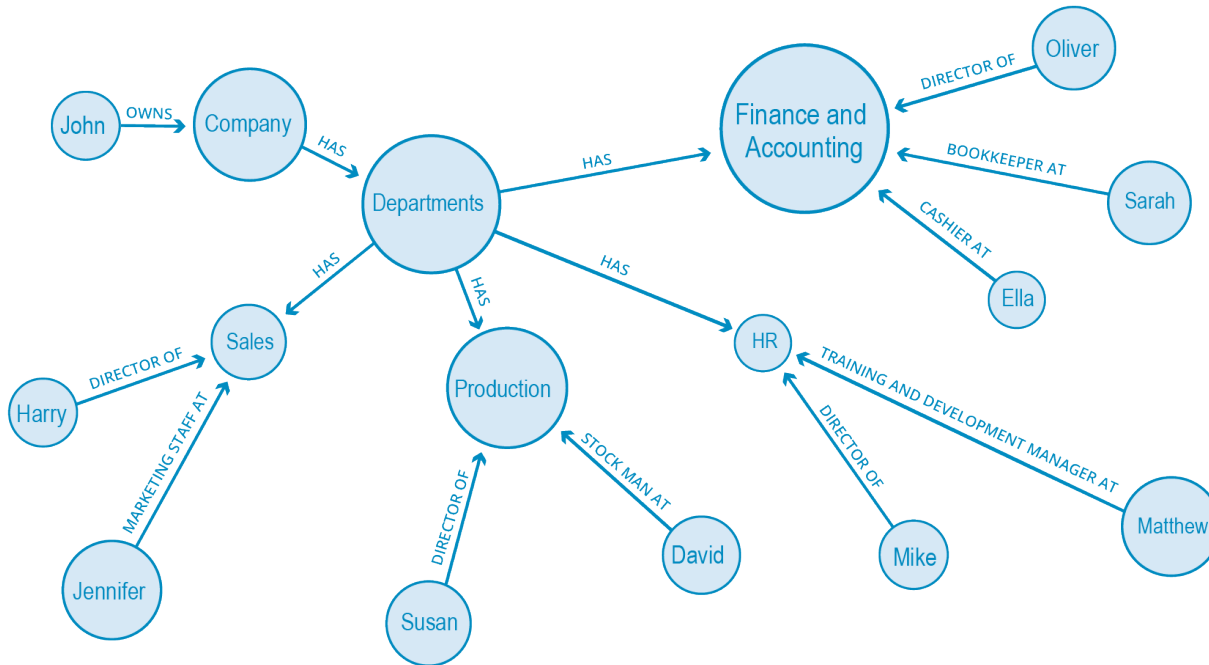
Esto implica:

Gran número de Joins

Joins recursivos para representaciones de árboles o jerarquías.

Esquema cambiante de las relaciones (frecuentes reorganizaciones).

¿Cómo analizamos las bases de grafos?



Las bases de grafos se centran en el **recorrido de relaciones**. En lugar de usar JOIN como en SQL, los grafos navegan las conexiones entre nodos directamente:

“Seguí este camino hasta encontrar lo que buscás”

- Esto permite **consultas más rápidas y naturales** en datos con muchas relaciones.

Bases de Datos Orientadas a Grafos





neo4j



- Neo4j es un sistema de base de datos orientado a grafos de código abierto
- Está desarrollado en Java
- Utiliza como modelo de datos, los grafos basados en propiedades

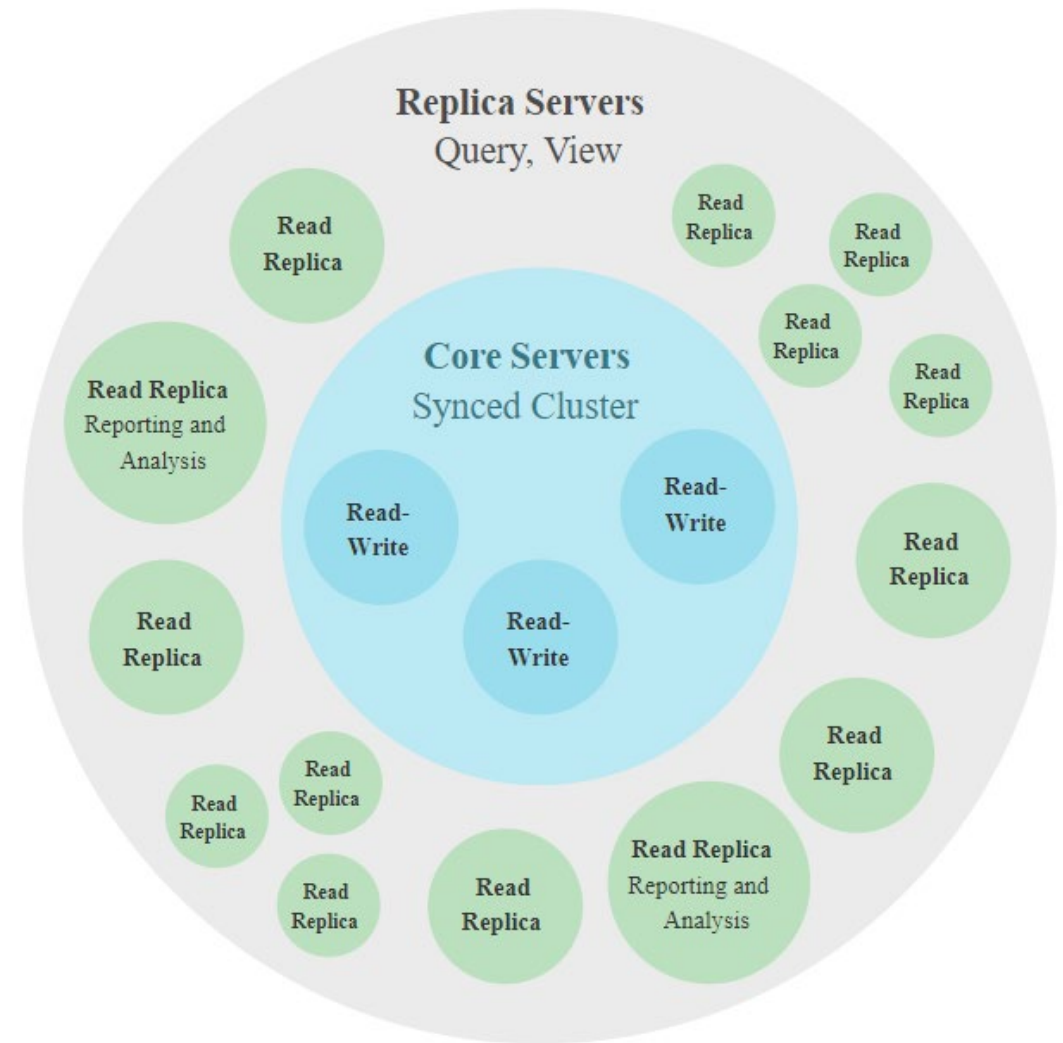
Es una de las pocas bases de datos orientadas a grafos que cuentan con un lenguaje de consulta propio: **Cypher**.

- El DBMS puede administrar un servidor independiente o un grupo de servidores en un Clúster (una instancia de Neo4j es un proceso de Java que ejecuta el código del servidor Neo4j).
- El contexto de ejecución es un entorno de ejecución para una solicitud (una consulta, una transacción o una función o procedimiento interno).

- **Core Servers**
 - Son los **nodos principales** del clúster.
 - Están **sincronizados entre sí** y **pueden realizar lecturas y escrituras** (Read-Write).
 - Mantienen **consistencia fuerte** (usando Raft, un algoritmo de consenso).
 - Se necesita una mayoría para confirmar una operación de escritura.
 - Ejemplo: Si hay 3 core servers, al menos 2 deben confirmar una escritura.
- **Read Replica Servers**
 - Solo pueden **leer** datos (Read Replica) → no escriben.
 - Pueden estar **geográficamente distribuidos**.
 - Se usan para:
 - Escalar la lectura
 - Generar reportes
 - Analítica
 - Balanceo de carga
 - Se sincronizan periódicamente con los Core Servers, pero no participan en el consenso.

Las **Read Replicas** en Neo4j se comportan más como un modelo **eventualmente consistente**.

- Aunque el core es **fuertemente consistente**, las réplicas **pueden recibir datos con cierto retraso**. Por eso podrías verlas como una mezcla inteligente entre **CA para core** y algo de **AP para lecturas distribuidas**.



Clustering

- Debemos tener relativamente pocas máquinas en un clúster Core Server esto se calcula con la fórmula

$$M = 2F + 1$$

- donde M es el número de servidores centrales necesarios para tolerar fallas y F el número de cores fallidos. (Ej. para tolerar dos servers fallidos, necesitamos un clúster de cinco Cores).
- También es posible crear un clúster que consta de solo dos núcleos. Sin embargo, ese clúster no será tolerante a errores. Si uno de los dos servidores falla, el servidor restante pasará a ser de solo lectura.



Cypher

- Su sintaxis utiliza un estilo **ascii-art** para hacer los comandos muy intuitivos.
- Mientras los nodos se representan con círculos y las relaciones con flechas, su representación en ascii-art consiste en poner nodos entre paréntesis y relaciones etiquetadas por corchetes:

`(nodo)-[:RELACIÓN]->(nodo)`

- Las propiedades de los nodos o las relaciones se indican con una estructura parecida a un diccionario:

`(nodo {name:'Oscar', surname:'Garcia'})`

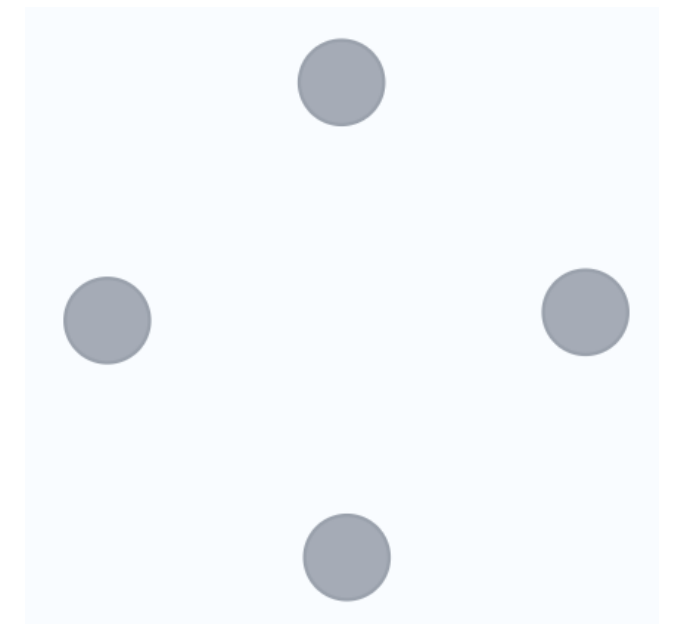
- Y las etiquetas se indican después de definir la variable:

`(nodo:Person {name:'Oscar', surname:'Garcia'})`

Cypher

- Empezamos por crear un nodo:
CREATE (os {name:'Óscar',
surname:'García'})
- Veamos si se ha creado:
MATCH (n) **RETURN** n
- Creemos un par de nodos más y
visualicémoslos:
va -> name: Vanessa, surname:
Gómez
co -> name: Coffee, type: Drink
na -> name: Napolitana, type:
Dessert
- ¿Como será la carga múltiple?

- **CREATE**
(va {name:'Vanessa',
surname:'Gómez'}),
(co {name:'Coffee',
type:'Drink'}),
(na {name:'Napolitana',
type:'Dessert'})



Cypher

- Para actualizar un nodo utilizamos la clausula **SET**:

```
MATCH (p:Person {name: 'Óscar'})  
  SET p.age = 35  
  RETURN p
```

- RETURN p: devuelve el nodo actualizado
- Se puede personalizar esta consulta para actualizar cualquier otra propiedad del nodo.
- Para actualizar múltiples propiedades, se debe agregar más cláusulas SET.
- Y si necesitas actualizar varios nodos al mismo tiempo, puedes ajustar la parte MATCH de la consulta para seleccionar los nodos deseados.

Cypher

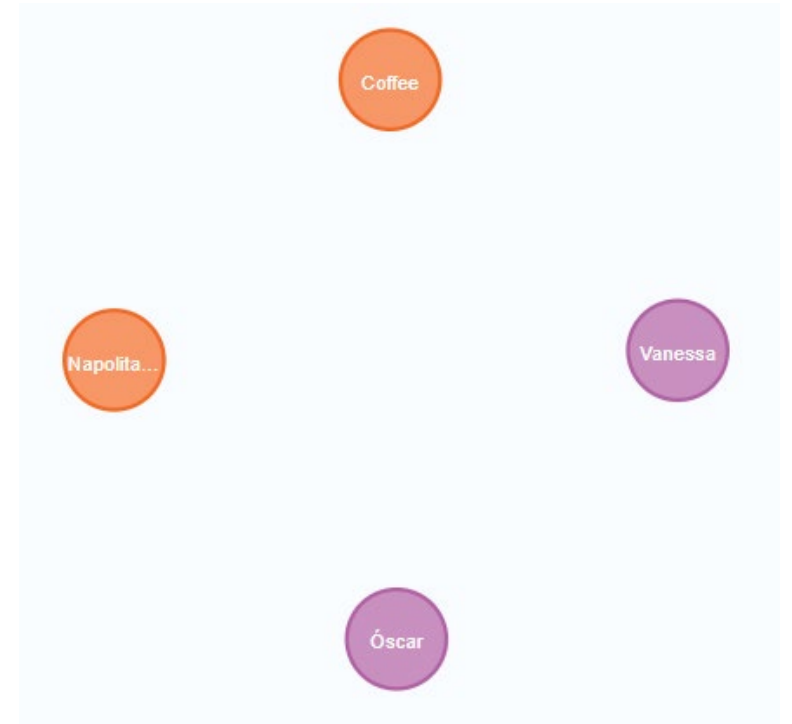
1. Vamos a añadir algunas etiquetas (ejecutar cada línea por separado):

```
MATCH (os  
  {name:'Óscar'})  
SET os:Person
```

```
MATCH (va  
  {name:'Vanessa'})  
) SET va:Person
```

```
MATCH (co  
  {name:'Coffee'})  
SET co:Food
```

```
MATCH (na  
  {name:'Napolita  
na'}) SET  
na:Food
```



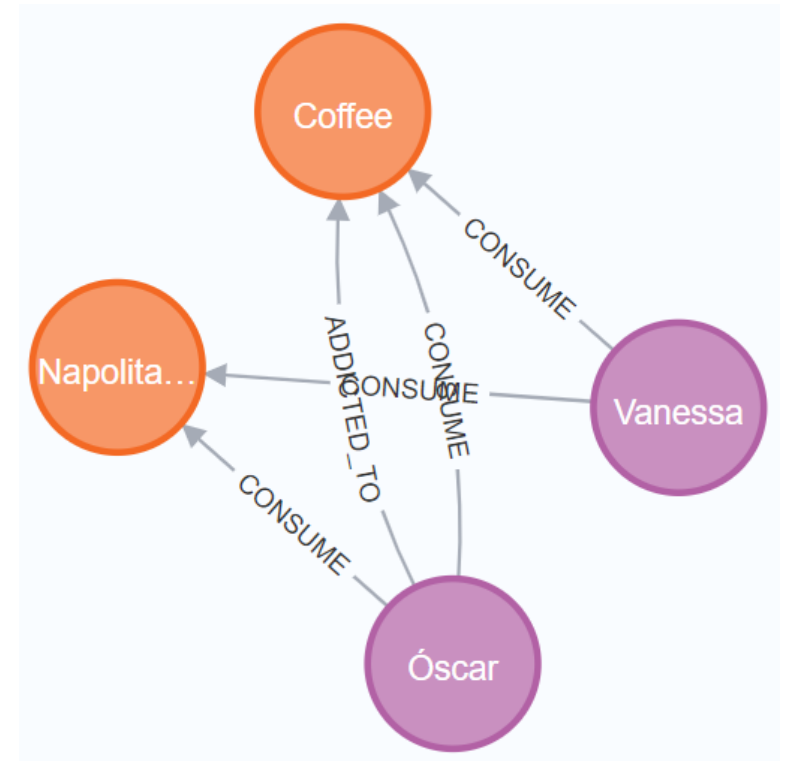
Cypher

- Ahora vamos a establecer una relación entre dos nodos

```
MATCH (o
      {name:'Óscar'}), (c
      {name:'Coffee'})
CREATE (o)-
[:ADDICTED_TO]-
>(c)
```

- Podemos crear relaciones entre nodos de forma simultánea:

```
MATCH (p:Person),
      (f:Food)
CREATE (p)-
[:CONSUME]->(f)
```



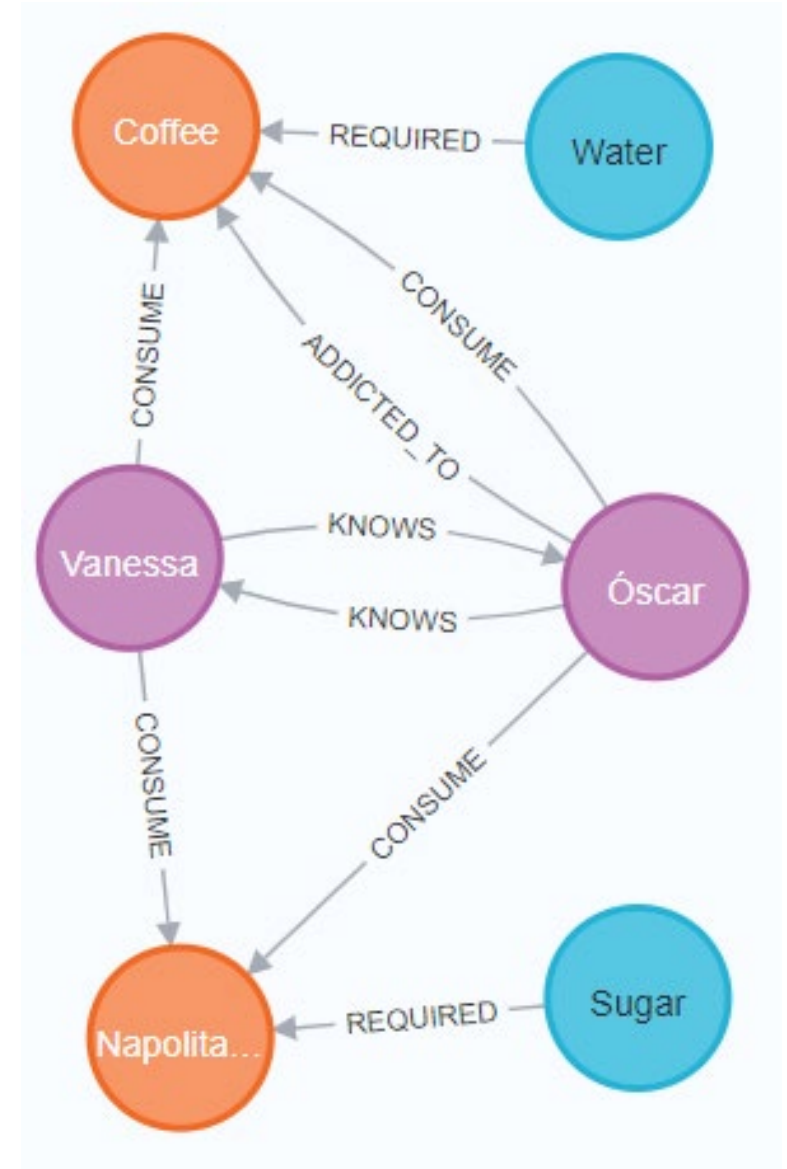
Cypher

- Creemos una relación bidireccional:

```
MATCH (a:Person), (b:Person)
WHERE a.name = 'Óscar' AND
b.name = 'Vanessa'
CREATE (a)-[:KNOWS]->(b), (b)-[:KNOWS]->(a)
```

- También podemos crear un par de nodos y relacionarlos en un sólo bloque:

```
CREATE
  (sug:Ingredient
   {name:'Sugar', type:'Liquid'}),
  (wat:Ingredient
   {name:'Water', type:'Liquid'})
WITH sug, wat
MATCH (na
 {name:'Napolitana'}), (co
 {name:'Coffee'})
CREATE
  (sug)-[:REQUIRED]->(na),
  (wat)-[:REQUIRED]->(co)
```



Cypher

Algunas Alternativas de visualización

- Todos los nodos con la misma etiqueta:
MATCH (n:Person) RETURN n
- Un par de atributos en forma tabular:
**MATCH (n:Person) RETURN n.name AS Names,
n.surname AS Surnames**
- Query relacional:
**MATCH (o {name:'Óscar'})-[:CONSUME]->(f) RETURN
f**
- La misma query pero solo un atributo en formato
tabular:
**MATCH (o {name:'Óscar'})-[:CONSUME]->(f) RETURN
f.name AS Food**

Un par de queries más. ¿Qué hacen?:

```
MATCH (o {name:'Óscar'})-[r]-(f) RETURN type(r)
MATCH (n:Person) WHERE n.name = 'Vanessa'
RETURN n.surname AS Surname
```


Cypher

Para eliminar podemos usar las cláusulas **DELETE** o **DETACH DELETE**

Eliminar un nodo:

```
MATCH (p:Person {name: 'Alice'})  
DELETE p
```

Eliminar una relación:

```
MATCH (:Person {name: 'Alice'})-[r:KNOWS]-()  
DELETE r
```

Eliminar un nodo y todas sus relaciones, y relaciones entrantes y salientes:

```
MATCH (p:Person {name: 'Alice'})  
DETACH DELETE p
```

Eliminar completamente nuestro grafo:

```
MATCH (n)  
DETACH DELETE n
```